

做饭灵感app-SDD V1.0

我们抛弃复杂的微服务和传统数据库，采用 **Serverless（无服务器）+ AI 驱动** 的极简架构，优先保证核心流程跑通，同时留出后续扩展接口。

系统设计文档 (SDD) - 极简 MVP 版

一、核心技术栈选型

- 前端框架**：Next.js (React) + Tailwind CSS。适合快速构建界面，自带路由，可以直接在 Vercel 免费部署。
- 后端服务**：Next.js API Routes。直接在前端项目中写后端逻辑，省去额外部署服务器的麻烦。
- AI 大模型引擎**：
 - 视觉模型**：用于拍照识图（如 Minimax Vision 或 Claude 3.5 Sonnet）。
 - 文本模型**：用于菜谱推荐和步骤生成（如 Minimax 或 Claude 3 Haiku，强调响应速度）。
- 数据存储**：浏览器 `localStorage`。满足 MVP 阶段的“弱状态管理”需求，直接将上一次的识别结果存在用户本地，0 成本（）。

二、核心业务数据流 (3 步核心路径)

为了满足“3步以内完成决策”，系统交互需高度扁平化：

1. 拍照与识别流 (Image -> JSON)

- 前端**：调用设备摄像头拍照，压缩图片（减少上传时间，保障 ≤ 3 秒 响应），转为 Base64 格式发送至后端 API。
- 后端 API**：将图片发送给 AI 视觉大模型。
- Prompt 策略**：强制 AI 输出纯 JSON 格式的食材数组。
- 前端处理**：接收 JSON，存入本地 `localStorage`，并在界面渲染，允许用户增删改（）。

2. 菜谱推荐流 (Ingredients -> Recipe List)

- 前端：读取本地食材列表，发送至推荐 API。
- 后端 API：

分为两步处理，分别由 AI 与本地规则完成：

1) 候选菜谱生成 (AI)

- 调用 AI 文本大模型，根据输入食材生成一组候选菜谱（约 15-20 个）。
- Prompt 策略：仅要求 AI 返回“菜名列表”，不参与匹配度计算与排序。

2) 推荐排序与筛选 (本地逻辑)

- 后端基于预定义规则对候选菜谱进行匹配度计算与排序。
- 匹配度计算方式：
 - 根据“已有食材数 / 菜谱所需食材总数”计算基础匹配分数
- 排序规则：
 - 优先展示“完全匹配”的菜谱
 - 次级展示“缺少少量食材”的菜谱
- 输出结构：

返回排序后的前 10 条推荐结果，并附带结构化字段：

- matchStatus：匹配状态 (perfect / partial)
- matchTag：展示文案 (如“食材已备齐” / “缺少土豆”)
- missingIngredients：缺失食材列表
- 响应优化：

推荐结果以轻量级 JSON 返回，保障 ≤2秒 响应时间。

3. 菜谱详情生成流 (Recipe Name -> Details)

- 前端：用户点击列表某一项，携带“菜名”和“现有食材”跳转详情页。
- 后端 API：调用 AI 生成具体步骤。
- UX 优化：使用 流式输出 (Streaming) 技术。不要等整篇做法生成完才展示，而是像打字机一样逐字显示，彻底消除用户的等待焦虑感 ()。

三、核心数据结构定义 (供 AI 编程参考)

你可以直接把以下 JSON 结构丢给 Claude Code，告诉它“这是我们前后端交互的数据契约”：

4. 食材列表 (Ingredient List)

JSON

代码块

```
1  [  
2    { "id": "1", "name": "西红柿", "category": "蔬菜" },
```

```
3    { "id": "2", "name": "鸡蛋", "category": "肉蛋奶" }
4  ]
```

5. 推荐结果列表 (Recommendation List)

JSON

代码块

```
1  [
2    {
3      "id": "r1",
4      "recipeName": "西红柿炒鸡蛋",
5      "matchStatus": "perfect",
6      "matchTag": "食材已备齐",
7      "missingIngredients": []
8    },
9    {
10     "id": "r2",
11     "recipeName": "土豆牛腩",
12     "matchStatus": "partial",
13     "matchTag": "缺少土豆、牛腩",
14     "missingIngredients": ["土豆", "牛腩"]
15   }
16 ]
```

matchStatus 与 matchTag 由后端规则计算生成，不依赖 AI 输出

四、性能优化与兜底策略 (稳定性保障)

- **超时兜底**：如果 AI 接口请求超过设定的 3 秒 或 2 秒，前端直接终止请求，并弹出“AI 正在打盹，请重试”的提示按钮。
- **识别错误兜底**：提供一个显眼的“手动添加/修改食材”按钮，这是极其重要的防线。
- **扩展性预留**：虽然当前数据存在 `localStorage`，但在代码结构上要封装一个 `StorageService` 类。未来需要做“菜谱收藏”或“动态偏好”时 ()，只需将这个类的内部实现替换为云端数据库（如 Supabase）即可，无需修改业务代码。